Processos de Software

Por que processo importa, e os modelos de ciclo de vida

DIM0510 — Turma 01 · Aula 01 (Sprint 0)

Prof. Fernando · UFRN · 2026.2

Roteiro de hoje

Bloco	O que vemos
Como o curso funciona	Sprints, avaliação, provas e o que entregar em 11/09
A Sprint 0	Visão do produto, MVP e acordo de processo
O que é processo, e por que importa	Definições, a norma e dois casos
Modelos de ciclo de vida	Cascata, V, espiral, iterativo e RUP
Fechamento	Tarefas para segunda

> A pergunta que atravessa a aula: por que uma equipe entrega e outra sofre, com as mesmas pessoas e a mesma tecnologia?

O combinado

Todo o material da disciplina está **público no GitHub** desde hoje.

```
github.com/fmarquesfilho/processos-2026-2
```

- Plano de curso, cronograma aula por aula, sistemática de avaliação e rúbricas
- O guia da primeira entrega, com templates e exemplos

> A avaliação da aprendizagem está detalhada em `docs/AVALIACAO.md`, e os critérios de cada entrega em `docs/RUBRICAS.md`.

Como o semestre funciona

Sprint 0 (4 semanas) + **3 sprints de projeto** + **bloco final**

Toda sprint segue o mesmo ciclo — a Sprint 1 como exemplo:

Quando	O quê
14 e 16/09	Aulas presenciais, com o conteúdo da sprint
21/09	Encontro online, para dúvidas do projeto
28 e 30/09	Apresentações: uma sessão online, uma em sala
02/10	**Entrega**, sexta-feira, 23:59

> A Sprint 0 foi estendida em duas semanas. Os feriados deslocam esse ritmo em cada sprint. As datas efetivas estão em `docs/CRONOGRAMA.md`.

Avaliação

Unidade	Composição	Fecha em
U1	Sprint 0 (20%) + Sprint 1 (40%) + Sprint 2 (40%)	23/10
U2	Sprint 3 (60%) + Prova (40%)	30/11
U3	Entrega final (60%) + Prova (40%)	11/12

Cada sprint: **50%** entrega técnica · **30%** atividade no repositório · **20%** comunicação

> Nota final é a média das três unidades. A U1 não tem prova e fecha em 23/10, então vocês conhecem o próprio desempenho antes de decidir sobre a segunda prova.

Duas provas, vale a maior

Data	Prova	Conteúdo
21/10	Prova escrita	Sprints 0 a 2
30/11	Prova de reposição, **opcional**	Cumulativa, Sprints 0 a 3

- Individuais, questões fechadas, no Multiprova, em laboratório
- Consulta permitida a **uma folha A4 manuscrita**, frente e verso
- Quem não fizer a segunda fica com a nota da primeira

> Essa mudança veio da turma: a prova estava tarde demais para dar chance de recuperação. Agora a primeira nota sai em outubro.

Os 30% do repositório

Ao fim de cada sprint, o que conta:

- CI verde no momento do prazo
- Commits distribuídos ao longo da sprint, não todos na véspera
- Pull requests com revisão de outro integrante
- Issues movimentadas no quadro

Há também um **fator de participação individual**.

> Neste curso isso não é só nota: é o próprio objeto de estudo. Vocês vão medir o fluxo do próprio time e propor melhorias com base nesses dados.

Datas que importam

Data	Compromisso
11/09	Entrega da Sprint 0 — proposta e acordo de processo
02/10	Sprint 1 — fluxo de trabalho
21/10	Prova escrita, presencial, em laboratório
23/10	Sprint 2 — automação da entrega
20/11	Sprint 3 — fluxo de valor e qualidade
30/11	Prova de reposição, opcional
11/12	Entrega final

> Cronograma completo, aula por aula, em `docs/CRONOGRAMA.md`.

O que você entrega em 11/09

1. Repositório **público** com `README.md` descrevendo produto, problema e equipe
2. Backlog no GitHub Projects: mínimo 10 itens, ao menos 3 estimados e priorizados
3. `docs/proposta.md` — visão, MVP, backlog e **acordo de processo**
4. Vídeo de 5 minutos

Stack tecnológico **livre**. Grupos de **1 a 4** integrantes, formados até 11/09.

> Aqui o processo pesa tanto quanto o produto. O guia completo está em `docs/SPRINT-0.md`.

Visão do produto

Antes de escrever código, responda por que o produto existe. Template obrigatório:

```
Para [usuários-alvo]
Que [problema ou necessidade]
O [nome do produto] é um [categoria]
Que [benefício principal]
Diferente de [alternativa existente]
Nosso produto [diferencial único]
```

> Se a equipe não consegue preencher isso, ainda não tem projeto — tem só uma ideia.

MVP e acordo de processo

MVP

O escopo mínimo que entrega valor. Declare explicitamente o que fica **fora** — é isso que protege o prazo.

Acordo de processo

Cadência, cerimônias, Definição de Pronto, papéis, ferramentas e WIP limits.

> O acordo de processo é o entregável que distingue esta disciplina. Ele vai ser confrontado com a realidade nas sprints seguintes — pode mudar, mas não pode faltar.

O que precisa estar no acordo de processo

Item	Pergunta que ele responde
Cadência	De quanto em quanto tempo a equipe se sincroniza e entrega?
Papéis	Quem decide prioridade? Quem revisa? Quem publica?
Definição de Pronto	Que condições um item precisa cumprir para sair do quadro?
Fluxo do quadro	Que colunas existem, e o que autoriza mover um cartão?
Limite de trabalho em curso	Quantos itens podem estar abertos ao mesmo tempo?

> Uma Definição de Pronto sem critério verificável — "está bom" — não é definição. "Testes passando, revisado por outro integrante e implantado no ambiente de homologação" é.

O que é um processo de software?

> Um **processo de software** é o conjunto de atividades, métodos e práticas usados para desenvolver e manter software.

- Quem faz o quê, quando e como
- Como a equipe se organiza e se comunica
- Como o trabalho flui do "precisamos disso" até "está em produção"

O que processo não é: burocracia, documentação por documentação, nem o oposto de agilidade.

Os cinco elementos de qualquer processo

Elemento	Definição	Exemplo
Atividade	Trabalho que transforma entradas em saídas	Revisar um pull request
Artefato	Produto de trabalho, entregue ou consumido	Backlog, código, relatório de teste
Papel	Conjunto de responsabilidades, não um cargo	Product Owner, revisor, mantenedor
Marco	Ponto de verificação com critério de saída	Fim da sprint, aprovação de arquitetura
Política	Regra que autoriza ou impede uma transição	"Não integra sem CI verde"

> Descrever o processo do seu grupo é preencher essas cinco linhas. Se um artefato não é consumido por nenhuma atividade, ele é desperdício — e vocês vão medir isso na Sprint 3.

Processo, método e ferramenta

Três coisas que se confundem no dia a dia:

Camada	O que é	Exemplo
Processo	O arranjo geral de atividades e responsabilidades	Desenvolvimento iterativo em sprints
Método	Uma forma específica de realizar parte dele	Scrum, XP, Kanban, revisão por pares
Ferramenta	O que apoia a execução	GitHub Projects, Actions, SonarQube

> Adotar a ferramenta sem o método, e o método sem entender o processo, é o erro mais comum das equipes.

O que a norma diz

A **ISO/IEC/IEEE 12207** descreve os processos do ciclo de vida de software, em quatro grupos:

Grupo	Contém
Acordo	Aquisição e fornecimento
Organizacionais	Gestão de portfólio, infraestrutura, recursos humanos, qualidade
Gestão técnica	Planejamento, controle, decisão, risco, configuração, medição
Técnicos	Requisitos, arquitetura, projeto, implementação, integração, verificação, transição, operação, manutenção, descontinuação

> Reparem que **manutenção e descontinuação** estão lá — a maior parte do custo de um software vem depois da primeira entrega.

Prescritivo e adaptativo

Prescritivo

Define de antemão as fases, os artefatos e os critérios de passagem. Prevê para controlar.

Cabe quando: requisitos estáveis, regulação, contrato fechado.

Adaptativo

Define o ritmo e os pontos de inspeção; o conteúdo é decidido a cada ciclo. Aprende para corrigir.

Cabe quando: escopo incerto, cliente disponível, entrega frequente.

> Não é uma escolha binária, e nenhum dos dois dispensa rigor na disciplina da equipe para conduzir o processo.

Por que estudar processos?

O desafio da engenharia de software mudou de lugar:

```
Antes:  "Como escrever código?"
Hoje:   "Como organizar pessoas, ferramentas e práticas
         para entregar software que funciona?"
```

- Equipes distribuídas viraram norma
- Ciclos de entrega encurtaram até o deploy contínuo
- **IA gera código em escala industrial**

> Processo é o que diferencia uma equipe que entrega de uma que sofre. Vamos ver dois casos que mostram isso.

O caso XZ Utils — CVE-2024-3094

Engenharia social de três anos contra o open source

```
2021  "Jia Tan" começa a contribuir com patches legítimos
2022  Contas falsas pressionam o mantenedor — "o projeto está lento"
2023  Jia Tan vira co-mantenedor oficial do XZ Utils
2024  Backdoor nas versões 5.6.0 e 5.6.1
      → execução remota de código via SSH
      → CVSS 10.0, a nota máxima de severidade
```

> Descoberto **por acaso**: um engenheiro da Microsoft notou o SSH 500 ms mais lento que o normal.

XZ Utils — onde o processo falhou

As falhas

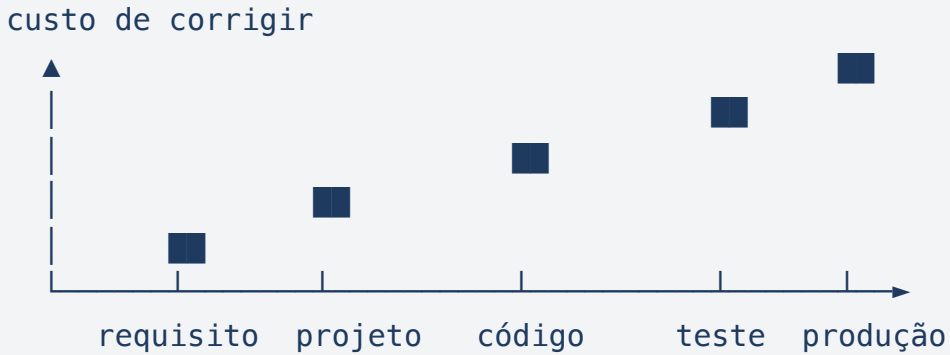
- Mantenedor solo, sobrecarregado
- Revisão insuficiente: 1 pessoa = 0 revisão
- Acesso concedido por conveniência
- Tarball diferente do Git

O que teria impedido

- Revisão obrigatória, 2 ou mais revisores
- Builds reproduzíveis: tarball = git
- Rotação e auditoria de mantenedores
- Diversidade de contribuidores

> Nenhuma dessas defesas é tecnologia. Todas são **processo**. É por isso que processo salva infraestrutura crítica.

O custo de descobrir tarde



Boehm observou, nos anos 1980, que o custo de corrigir um defeito cresce por ordem de grandeza a cada fase em que ele passa despercebido.

> Esse princípio permanece, e explica quase tudo que a engenharia de software hoje recomenda: revisão, teste automatizado, integração contínua e entrega frequente existem para **encurtar o tempo entre errar e descobrir**.

Modelos de ciclo de vida

Um **modelo de ciclo de vida** organiza as atividades do desenvolvimento no tempo: o que vem antes, o que vem depois, e o que se repete.

Vamos ver cinco, na ordem em que apareceram:

Cascata · **Modelo V** · **Espiral** · **Iterativo e incremental** · **RUP**

> Nenhum deles está "errado". Cada um responde a um contexto — e saber escolher é a competência que interessa.

O artigo mais mal interpretado da história

Royce, 1970 — *Managing the Development of Large Software Systems*

Royce **nunca** usou a palavra "cascata". O diagrama sequencial aparece na **página 2**, e logo em seguida ele escreve:

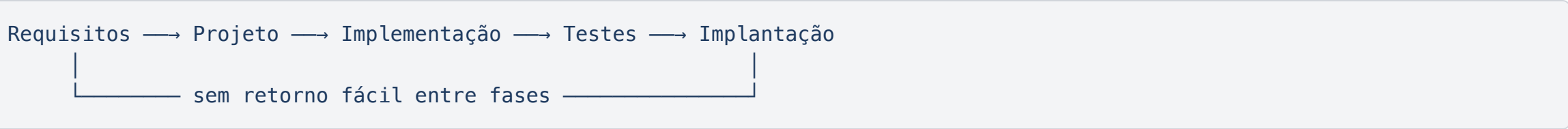
> *"I believe in this concept, but the implementation described above is risky and invites failure."*

O resto do artigo propõe iterações, protótipos e feedback entre fases.

O que aconteceu: o Departamento de Defesa dos EUA adotou o diagrama da página 2, ignorou as nove páginas de ressalvas, e isso virou o padrão MIL-STD-2167 em 1985.

Modelo Cascata

e como ficou conhecido depois que os militares o adotaram



Premissas, raramente verdadeiras: requisitos conhecidos e estáveis desde o início, erros encontrados na fase em que ocorrem, cliente que sabe exatamente o que quer.

> Onde ainda faz sentido: sistemas embarcados críticos, contratos com requisitos fixos, e situações em que o custo de falha é catastrófico.

Fases do Modelo Cascata

Fase	Entrega	Critério de saída
Requisitos	Especificação do que o sistema deve fazer	Documento aprovado pelo cliente
Projeto	Arquitetura e projeto detalhado dos módulos	Revisão de projeto aprovada
Implementação	Código e testes de unidade	Módulos compilando e testados
Testes	Integração, sistema e aceitação	Defeitos abertos abaixo do limite
Implantação	Sistema em operação, e manutenção	Aceite formal

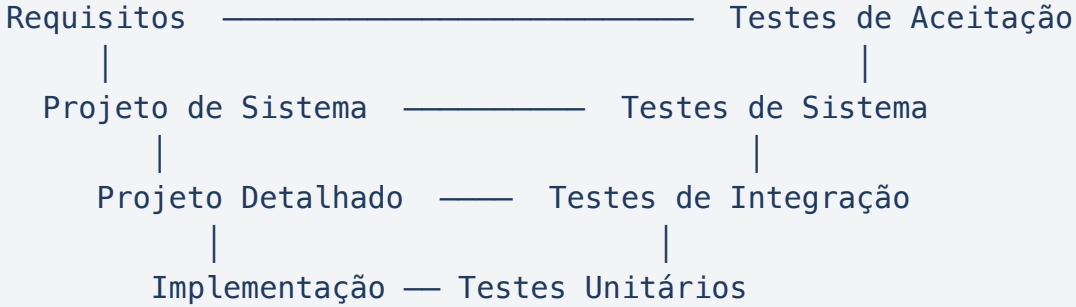
> Repare no que a coluna da direita tem em comum: **aprovação documental**. Numa cascata, o que autoriza avançar é um documento — não software funcionando. É essa a inversão que o ágil promove.

O cone da incerteza



No início do projeto, uma estimativa pode errar por um fator de quatro, para mais ou para menos. A incerteza só cai **à medida que se constrói**.

Modelo V — teste como espelho



Cada nível de definição ganha um nível de verificação correspondente, planejado **ao mesmo tempo**.

> Herda o problema da cascata, mas a ênfase em testes por nível é uma contribuição que sobreviveu — está viva na pirâmide de testes de hoje.

Verificação e validação

Duas perguntas diferentes, que o modelo V separa explicitamente:

	Pergunta	Como se faz
Verificação	Estamos construindo o produto **corretamente**?	Testes, revisão, análise estática, prova
Validação	Estamos construindo o **produto certo**?	Protótipo com usuário, teste de aceitação, piloto

> Um sistema pode passar em 100% dos testes e ainda ser inútil: verificado e não validado. É o defeito mais caro que existe, porque só aparece na entrega — e é por isso que os modelos seguintes trazem o usuário para dentro do ciclo.

Prototipagem

Construir algo incompleto de propósito, para aprender antes de comprometer.

Tipo	O que se faz com ele	Risco que ataca
Descartável	Joga fora depois de responder à dúvida	Requisito mal entendido, interface duvidosa
Evolutivo	Vira a base do produto, refinado a cada ciclo	Viabilidade técnica, desempenho

> Você pode trabalhar com protótipos usando dados mockados, uma vez que o foco principal é validar uma hipótese

A Espiral de Boehm (1988)

Cada volta da espiral tem quatro atividades:

1. **Definir objetivos** — o que queremos desta iteração?
2. **Analisar riscos** — o que pode dar errado? Prototipar.
3. **Desenvolver e verificar** — construir o incremento
4. **Planejar** a próxima volta

> A contribuição fundamental é o **risco como critério de decisão**. Antes de codificar, pergunte qual é o maior risco — e ataque ele primeiro.

Risco, com definição

> **Risco** é um evento incerto que, se ocorrer, afeta os objetivos do projeto.

Exposição = probabilidade × impacto. É isso que ordena a fila.

Resposta	O que significa	Exemplo
Evitar	Mudar o plano para eliminar o risco	Trocar a biblioteca sem manutenção
Mitigar	Reduzir probabilidade ou impacto	Protótipo da integração na primeira semana
Transferir	Passar o impacto adiante	Serviço gerenciado em vez de servidor próprio
Aceitar	Conviver, com plano de contingência	Documentar e monitorar

> Na Sprint 0 vocês registram os riscos do projeto. Na Sprint 3, revisitamos: quais se realizaram, e o que a equipe fez a respeito.

Iterativo e incremental

```
Iteração 1:  [Requisitos → Design → Código → Teste] → Incremento 1
Iteração 2:  [Requisitos → Design → Código → Teste] → Incremento 2
Iteração 3:  [Requisitos → Design → Código → Teste] → Incremento 3
```

São dois conceitos distintos: **iterativo** é refinar o mesmo artefato em ciclos; **incremental** é adicionar funcionalidade a cada ciclo.

> A vantagem decisiva: feedback do cliente a cada incremento, não só no final. Erros aparecem cedo, quando são baratos. É a base conceitual do ágil — e do formato de sprints deste curso.

A diferença, com um exemplo

Pintar um quadro:

```
Incremental  ▸  canto esquerdo pronto │ mais um pedaço │ quadro inteiro
Iterativo    ▸  esboço da tela toda   │ formas         │ acabamento
```

Software real usa os dois: cada sprint **acrescenta** funcionalidade e **refina** o que já existe.

> O erro que aparece nas apresentações: entregar só a camada de baixo — banco e API prontos, nenhuma tela. Isso não é incremento, é uma fatia horizontal que ninguém consegue usar nem avaliar. **Incremento é fatia vertical: chega até o usuário.**

RUP — Rational Unified Process



Quatro fases, cada uma com várias iterações, atividades em paralelo com intensidades diferentes. Pesado em documentação e artefatos UML.

> Importa como **ponte**: foi o RUP que levou iteração para dentro de organizações conservadoras, preparando o terreno para o ágil.

RUP: os marcos e o que sobreviveu

Fim da fase	Marco	Pergunta que precisa estar respondida
Concepção	Objetivos do ciclo	O projeto se justifica? Escopo e riscos identificados
Elaboração	Arquitetura do ciclo	A arquitetura é estável e os maiores riscos caíram?
Construção	Capacidade operacional	O sistema está pronto para os usuários piloto?
Transição	Liberação do produto	O usuário aceitou e a operação assumiu?

Das seis práticas que o RUP pregava, quatro são hoje consenso: **desenvolver iterativamente, gerenciar requisitos, verificar qualidade continuamente e controlar mudanças**.

> O que não sobreviveu foi o peso: dezenas de artefatos obrigatórios, produzidos independentemente de haver quem os lesse.

O que o ágil de fato mudou

Iteração já existia na espiral e no RUP. A ruptura de 2001 foi outra:

Deixou de ser	Passou a ser
Documento aprovado autoriza avançar	**Software funcionando** é a medida de progresso
Cliente presente no começo e no fim	Cliente presente em todo ciclo
Mudança é desvio a ser controlado	Mudança é informação nova, bem-vinda
Processo definido de fora para a equipe	Equipe ajusta o próprio processo, com evidência

Linha do tempo

```
1970  Royce — Cascata (mal interpretado)
1981  Modelo V — ênfase em verificação
1986  Takeuchi & Nonaka — "Scrum", artigo original na HBR
1988  Boehm — Espiral, orientado a risco
1996  RUP — iterativo em contexto corporativo
1999  Beck — Extreme Programming
2001  Manifesto Ágil — ruptura filosófica
2009  Flickr — "10+ deploys per day", nasce o DevOps
2018  Forsgren et al. — Accelerate e as métricas DORA
2024  DORA — o paradoxo da produtividade com IA
```

> Cinquenta e seis anos de tentativas de responder à mesma pergunta: como organizar o trabalho para entregar software que funciona.

Para segunda, 24/08

Tarefas

- Ir organizando para formar o grupo do projeto, com 1 a 4 integrantes (se quiser pode usar o canal do Discord pra achar colegas)
- Trazer uma ideia de projeto, ainda que crua

Leituras

- Manifesto Ágil · `agilemanifesto.org`

Referências da aula

Da aula

- Royce (1970) · Boehm (1988) — *A Spiral Model*
- ISO/IEC/IEEE 12207 — processos do ciclo de vida de software
- DORA — *Accelerate State of DevOps Report* · `dora.dev`

Livro-texto aberto

- Valente, *Engenharia de Software Moderna* · `engsoftmoderna.info`

Disciplina

- `github.com/fmarquesfilho/processos-2026-2`